Los procesos: Procesos y Bloque de control de procesos

Sistemas Operativos I

Oliver Díaz

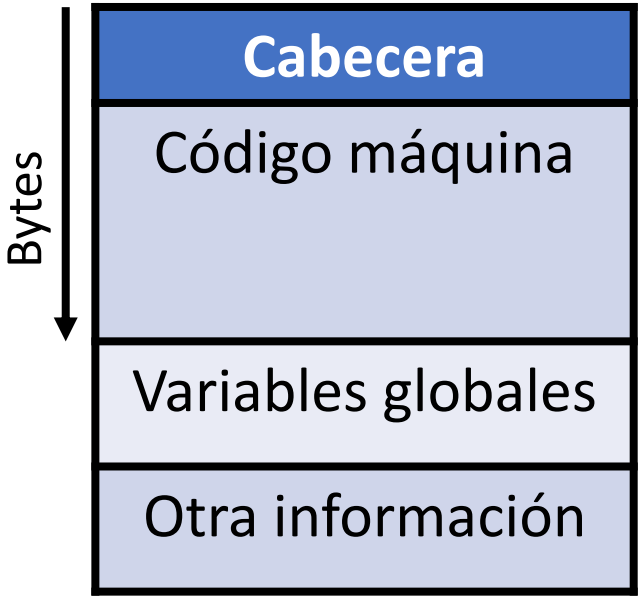
Grau d'Enginyeria Informàtica

Contenido de la unidad

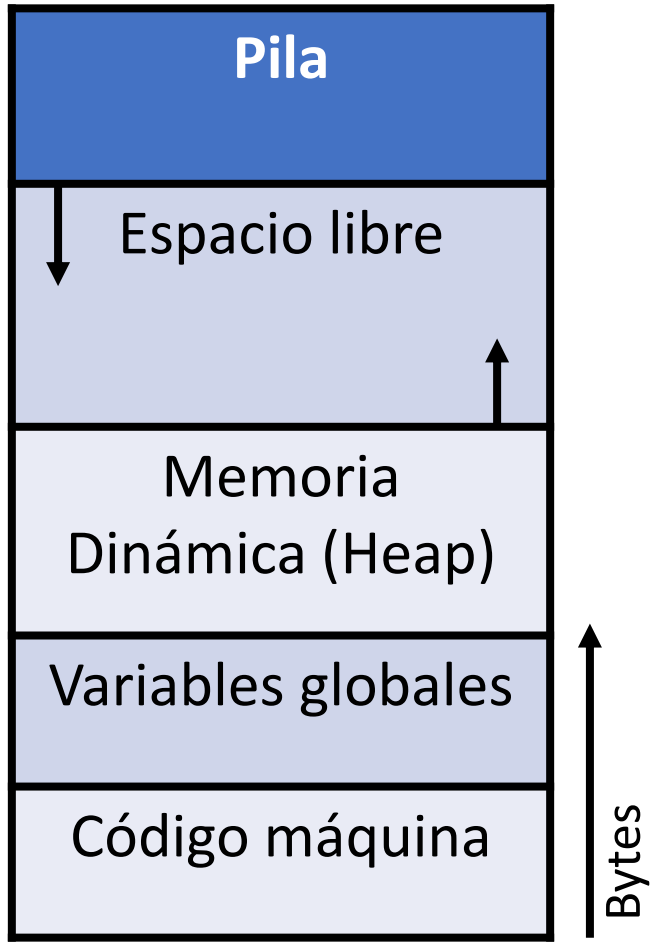
- Los procesos
- El bloque de control de procesos
- Llamadas a sistemas -> comunicación de procesos entre sí a través de API
- Creación de procesos en UNIX -> fork() y exec()
- Gestión de contextos
  - Redirección estándard a fichero (E/S) -> descriptores de fichero (dispositivos)
  - Tuberias/Canonades/Pipes

Los procesos

- **Proceso**: programa en ejecución



Estructura de un programa



Estructura de un proceso en memoria

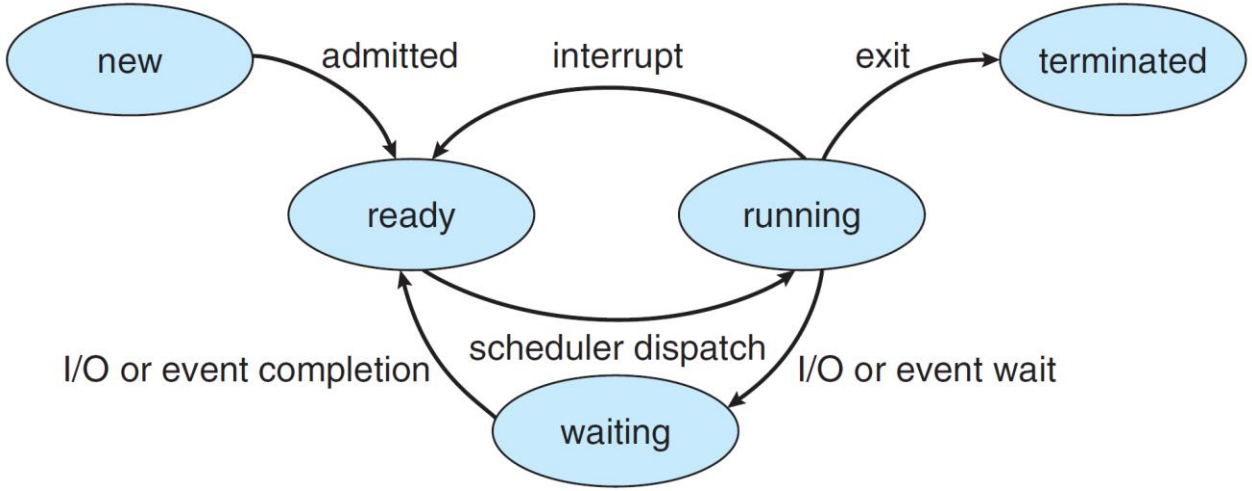
Bloque de control de procesos

- El **Bloque de Control de Procesos** (*Block Control Process* BCP) es una estructura de datos utilizada por el SO para representar y controlar cada uno de los procesos que se están ejecutando en el sistema.
- Contiene información detallada sobre cada proceso necesario para iniciar o reiniciar un proceso, etc.
- Utilizado por el SO para:
  - Planificar el uso de la CPU
  - Cambiar de contexto
  - Gestión de procesos en espera
  - Asignación de recursos
  - Terminación de procesos

Estado
Identificador (PID)
Prioridad
Tiempo CPU utilizado
Registro CPU
…

Estado de un proceso

- A medida que el proceso se ejecuta, su estado cambia. El **estado** de un proceso viene determinado por su actividad actual
  - Nuevo/*New*: ha sido creado, pero no admitido por el SO
  - Listo/*Ready*: en espera de que se le asigne tiempo CPU para ejecutarse
  - Ejecución/*Running*: esta siendo ejecutado por la CPU
  - En espera o bloqueado/*Waiting*: en espera de algun evento externo
  - Terminado o Salida/*Terminated* o *Exit*: finalización de ejecución



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Ejecución de los procesos

- **Sistemas monoprocesadores**
  - ¿Cómo se ejecutan varios procesos a la vez?
  - No puede, solo 1 proceso se ejecuta en un momento dado (“running”)
  - El resto de procesos pueden estar en modo “*Ready*” o “*Waiting*”
  - Se asigna un “*time slice*”/“ciclo de tiempo”/“rebanada de tiempo” de CPU a cada proceso (unos pocos milisegundos) -> ¡ilusión de paralelismo!

- **Sistemas multiprocesadores**
  - Paralelismo real
  - Cada procesador únicamente ejecuta 1 proceso en un momento determinado
  - El SO decide (mediante el planificador) que proceso se ejecuta en cada momento

Gracias

Los procesos: Interfaz de sistema

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Procesos y llamadas al sistema

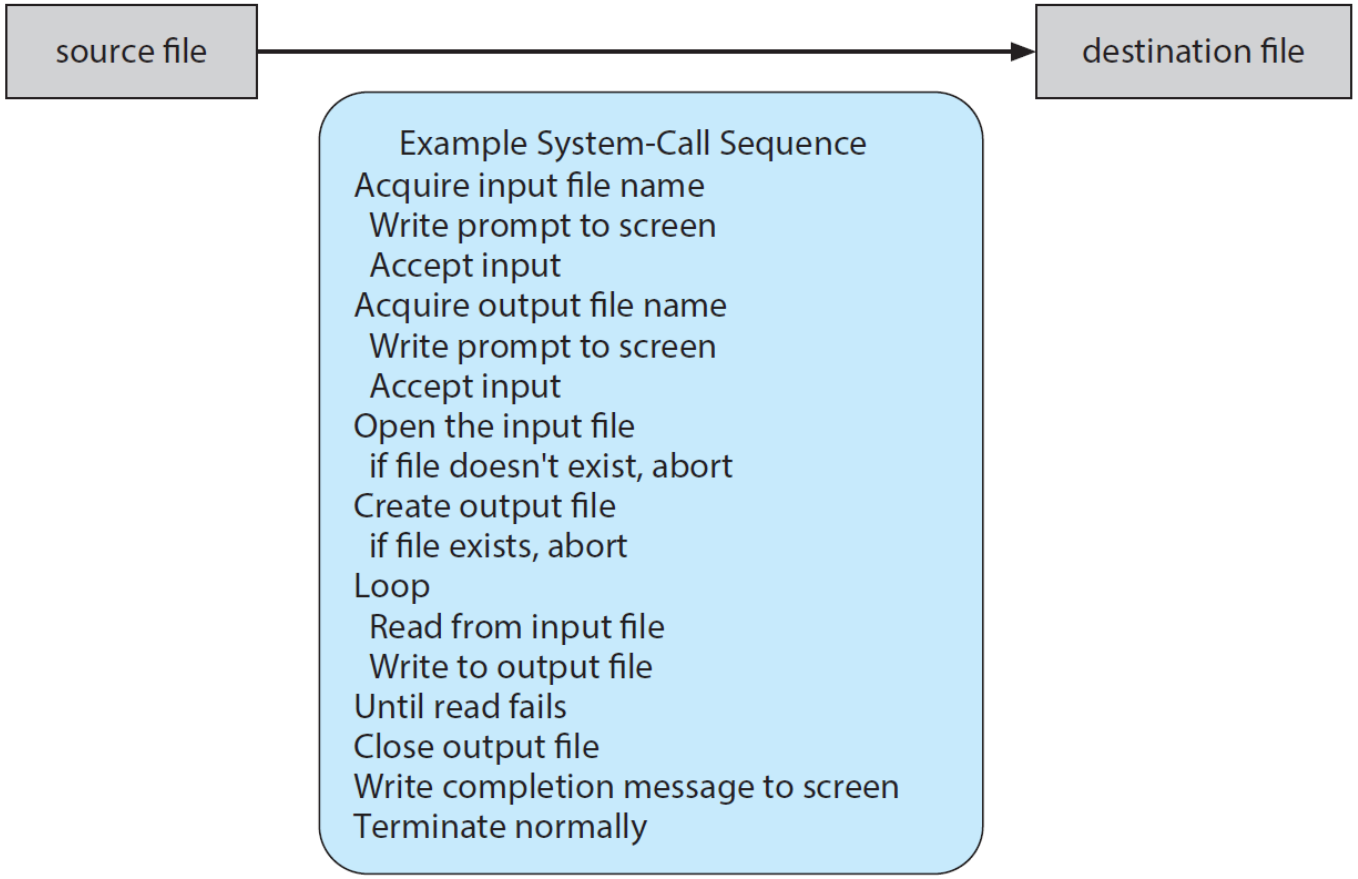
- Se require de **llamadas al sistema** para comunicar procesos entre sí
- "Solicitud que un programa de usuario hace al núcleo del SO para que realice una tarea en su nombre"
- Proporcionan una interfaz para los servicios disponibles por un SO.
- Generalmente disponibles como funciones escritas en alto nivel (C y C++), aunque ciertas tareas de bajo nivel pueden estar escritas en lenguaje ensamblador.
- Principalmente utilizadas por programas a través de una **interfaz de sistema** (*Application Programming Interface,* API), en lugar de una llamada directa al sistema (p. ej. POSIX). Ejemplo de API para UNIX:

```
#include <unistd.h>

ssize_t    read(int fd, void *buf, size_t count)
```

return value — function name — parameters

Llamada al sistema

```
cp source.txt destination.txt
```



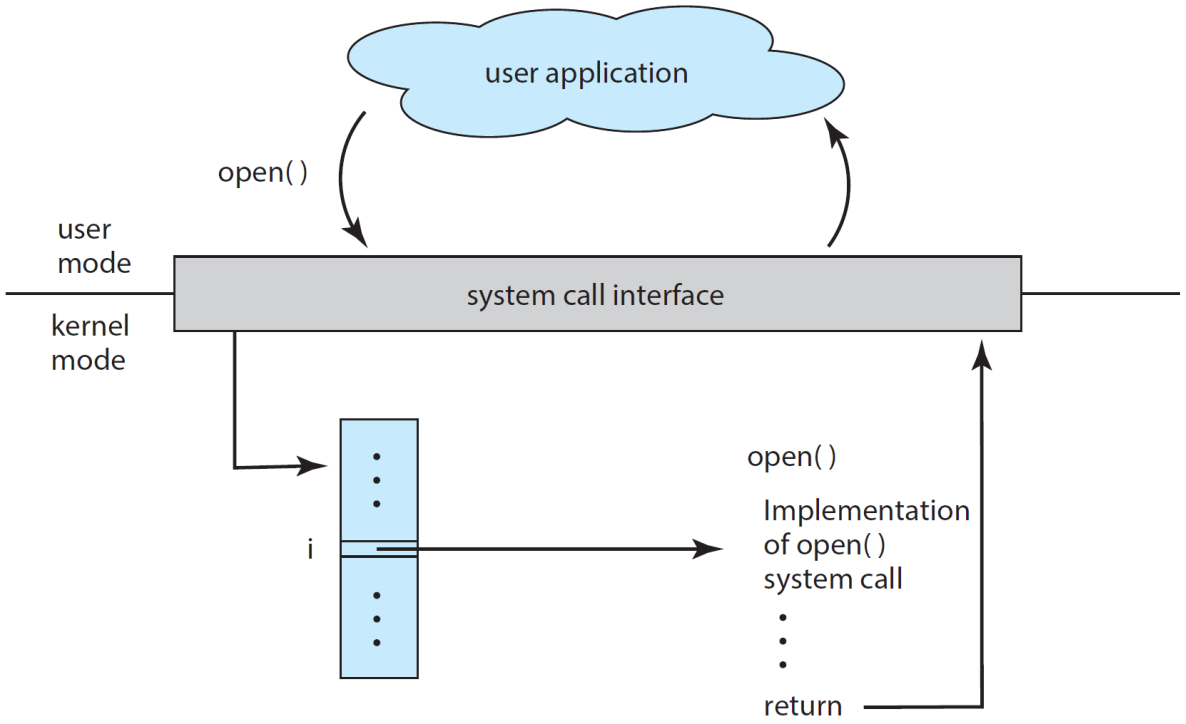
Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

POSIX

- **POSIX** (*Portable Operating System Interface for Unix*) es un conjunto de estándares para los SOs compatibles con POSIX (Linux, macOS, FreeBSD, Solaris,..).
- Aumenta la interoperabilidad y portabilidad de las aplicaciones y SOs
- Incluye:
  - Interfaces de programación de aplicaciones (API)
  - Shell de línea de comandos
  - Bibliotecas de C
  - Threads
  - …

Interfaz de sistema

- Una **interfaz de sistema** (API) es un conjunto de funciones, protocolos y herramientas que permiten que los procesos interactúen con el SOs o con otros procesos.



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Interfaz de sistema

- Define los **servicios que el SO proporciona a las aplicaciones**, y también define cómo los procesos pueden acceder a esos servicios y utilizarlos
  - Funciones para realizar operaciones de E/S
  - Administrar el almacenamiento de archivos
  - Gestión de procesos y recursos del sistema).
- Toda la funcionalidad esta implementada a nivel de núcleo. Los procesos acceden a estos servicios mediante llamadas al sistema.
- La interfaz de UNIX no ha cambiado mucho desde sus inicios (1973). Continua siendo muy utilizada hoy en día (potente, sencilla, portable)
- Al utilizar una interfaz de sistema, los desarrolladores de software pueden escribir aplicaciones que sean compatibles con el SO y con otros programas, sin tener que conocer los detalles internos de cómo funciona el SO o los demás programas.

Interfaz de sistema

¿Qué servicios ofrecen las **llamadas al sistema** de los SOs a los procesos? Entre otras cosas…

- **Gestión de procesos.** Administrar y controlar los procesos que se ejecutan en el sistema
  - ¿Puede un proceso crear otro proceso?
  - ¿Puede un proceso esperar a que otro proceso acabe de ejecutarse?
  - ¿Podemos parar o continuar la ejecución de un proceso?
- **Entrada-salida.** Controlar el flujo de datos entre los dispositivos de E/S (discos duros, teclados, pantallas, etc.) y la CPU.
  - ¿Cómo pueden los procesos comunicarse con los dispositivos conectados al ordenador?
  - ¿Pueden los procesos comunicarse entre si?
- **Gestión de memoria, gestión de red, etc.**

Interfaz de sistema

- Antiguamente, el núcleo creaba los procesos (¡fiables!)
- En la actualidad los mismos procesos crean otros procesos. P.ej:
  - Intérprete de comandas fue una de las primeras aplicaciones en crear procesos
  - Gestor gráfico de ventanas
  - Servidor web
  - Navegador web
  - Base de datos
  - Compiladores
  - Editores de texto
  - …
- Es necesario gestionar la seguridad.

¿Que beneficios tiene que un proceso cree otro proceso?

- Permite una mayor modularidad, flexibilidad, control, aislamiento y escalabilidad en el diseño de SOs y de software en general.
  - Si un proceso necesita realizar una tarea compleja o que requiere mucho tiempo, puede crear otro proceso para realizar una parte de esa tarea.
  - (**Modularidad**) Cada proceso puede realizar una tarea específica y comunicarse con otros procesos para completar una tarea más compleja.
  - (**Control**) Los programas de usuario pueden tener un mayor control sobre cómo se ejecutan las tareas en el SO.
  - (**Aislamiento**) Los procesos hijos están aislados del proceso padre y otros procesos
- Un navegador web puede utilizar Apps externas para dibujar una página por pantalla. Incrementa la funcionalidad del programa.
- Un servidor web puede invocar un proceso externo para dar formato a una página web antes de ser visualizada por el navegador de un dispositivo.

¿Que beneficios tiene que un proceso cree otro proceso?

- A los sistemas UNIX, a la línea de comandos, hay disponibles muchas aplicaciones que realizan tareas específicas.
- Las funcionalidades de las aplicaciones se pueden combinar utilizando la **redirección** y las **tuberías** para conseguir funcionalidades más potentes.

```
ls | wc -l
ls -l > fichero.txt
```

UNIX®
An Open Group Standard

Gracias

Los procesos: La gestión de procesos

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

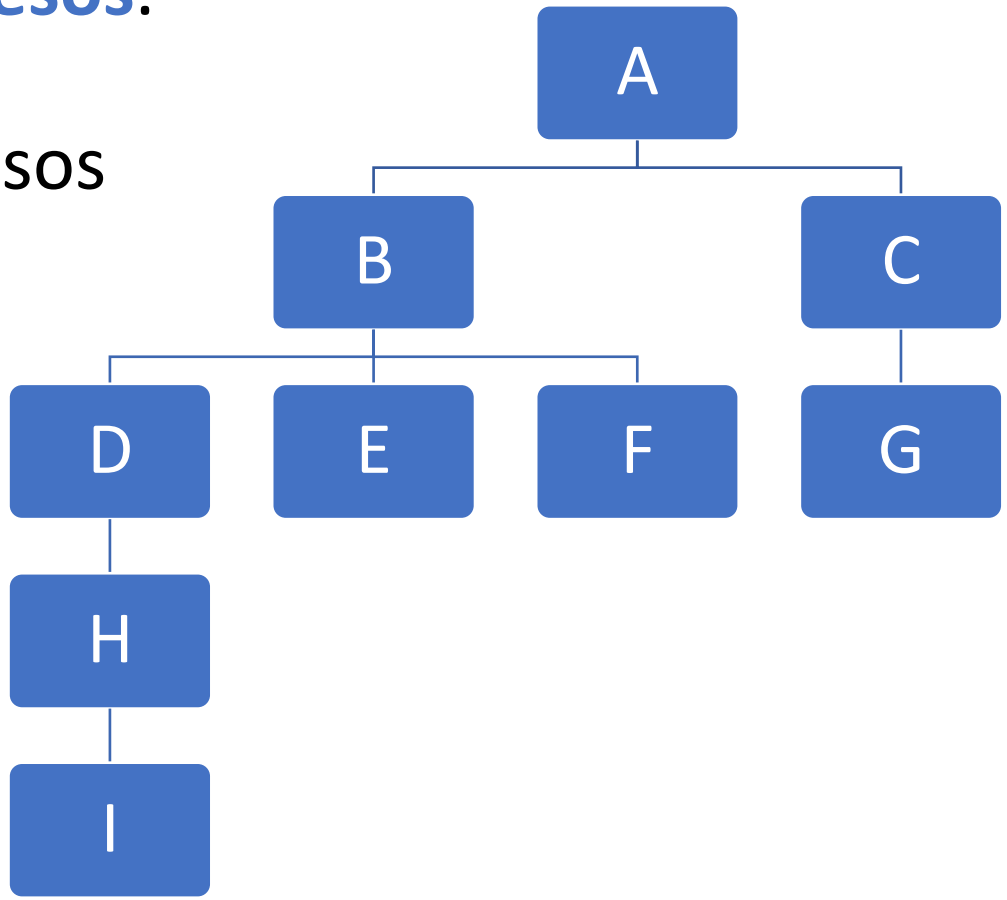
La gestión de procesos

- Parte fundamental de un SO que se encarga de administrar y controlar los procesos que se ejecutan en el sistema.
- El proceso que crea un proceso se llama **padre**, mientras que el proceso que es creado se llama **hijo**.
- Las tareas que realiza el núcleo del SO al crear un nuevo proceso son:
  1. Crea e inicializa el bloque de control de procesos
  2. Crea e inicializa el espacio de direcció de memoria
  3. Carga el programa a memoria
  4. Prepara el núcleo para comenzar a ejecutar el *main*

Jerarquia de procesos

- Dado que los procesos pueden crear otros procesos, el SO almacena información sobre esta **jerarquía de procesos**.

- Podemos ver la jerarquía de procesos mediante la instrucción `pstree`



Tipos de llamadas a sistema

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Gestión de procesos a Windows

- En SO Windows, la función que permite crear nuevos procesos es `CreateProcess`

```
// Start the child process
if (!CreateProcess(NULL,   // No module name (use command line)
    argv[1],        // Command line
    NULL,           // Process handle not inheritable
    NULL,           // Thread handle not inheritable
    FALSE,          // Set handle inheritance to FALSE
    0,              // No creation flags
    NULL,           // Use parent's environment block
    NULL,           // Use parent's starting directory
    &si,            // Pointer to STARTUPINFO structure
    &pi )           // Pointer to PROCESS_INFORMATION structure
)
```

- Los primeros dos argumentos permiten especificar el programa a ejecutar, así como los argumentos a pasar al *main*. El resto de argumentos están asociados a restringir el entorno en el que se ejecuta el proceso

Gestión de procesos a UNIX

- `fork()`
  - Crea un nuevo proceso **hijo**, independiente del **padre**
  - El hijo consiste en una copia del espacio de direcciones del proceso padre -> permite que el padre se pueda comunicar fácilmente con su hijo.
  - `fork()` devuelve dos veces, una para el padre (PID hijo) y otro para el hijo (0)

Padre

```
main
…

ret = fork()
ret==PID_hijo
```

Hijo

```
main
…

ret = fork()
ret==0
```

¿Cómo sabemos en un momento dado si somos el padre o el hijo?

Gestión de procesos a UNIX

- Ver código fork.c

```
int main(void)
{
  int ret;

  ret = fork();

  if (ret == 0) {  // fill
     printf("Soc el fill i el meu id es %d\n", getpid());
     return 0;
  } else { // pare
     printf("Soc el pare del proces %d\n", ret);
     return 0;
  }
}
```

Gestión de procesos a UNIX

- El proceso hijo creado con el `fork()` es independiente del proceso padre. Ver código fork-variables.c

```c
int main(void)
{
  int ret, a;

  a = 1;

  ret = fork();

  if (ret == 0) {  // fill
     printf("Soc el fill i el meu id es %d\n", getpid());
     printf("Fill a = %d\n", a); // s'imprimeix a = 1!
     return 0;
  } else { // pare
     a = 2;
     printf("Soc el pare del proces %d\n", ret);
     printf("Pare a = %d\n", a); // s'imprimeix a = 2!
     return 0;
  }
}
```

Fork-bomb

- Tipo de ataque informático que se utiliza para hacer que un SO se vuelva inutilizable mediante la creación de un gran número de procesos en un corto período de tiempo. El ataque funciona explotando la función `fork()`.
- Un proceso crea otro proceso, que a su vez crea otro proceso y así sucesivamente, de manera exponencial. Cada proceso creado consume una pequeña cantidad de recursos del sistema (RAM, CPU), pero en conjunto, pueden agotar rápidamente los recursos disponibles del sistema

```
0 #include <stdio.h>
1 #include <unistd.h>
2
3 int main(void)
4 {
5     while (1) { fork(); }
6 }
```

Gestión de procesos a UNIX

- **exec()**
  - Después de crear un proceso hijo con `fork()`, una de las llamada a sistema más común en UNIX es `exec()` para ejecutar un nuevo programa (¡No crea un nuevo proceso!)
  - Carga un archivo binario a memoria e inicia su ejecución --> Reemplaza la imagen del proceso por la especificada en `exec()`.
  - El padre se puede poner en `wait()` hasta que termine su hijo
  - Ver código fork-exec.c

```c
int main(void)
{
  int ret;
  char *argv[3] = {"/usr/bin/ls", "-l", NULL};

  ret = fork();

  if (ret == 0) {  // fill
     printf("Soc el fill i el meu id es %d\n", getpid());
     execv(argv[0], argv);
  } else { // pare
     printf("Soc el pare del proces %d\n", ret);
     return 0;
  }
}
```

¿Por qué UNIX utiliza 2 funciones?

- Al realizar el `fork()`, **el proceso hijo hereda el contexto del padre** (privilegios, ficheros abiertos, etc.)
- Después del `fork()`, y antes de hacer el `exec()`, se puede establecer un **nuevo contexto** para programa a ejecutar mediante llamadas al sistema.

```c
int main(void)
{
  int ret;
  char *argv[3] = {"/usr/bin/ls", "-l", NULL};

  ret = fork();

  if (ret == 0) {  // fill
     printf("Soc el fill i el meu id es %d\n", getpid());
     // Aqui s'estableix el nou context del proces fill!
     // Es fan les crides necessaries abans de fer exec
     execv(argv[0], argv);
  } else { // pare
     printf("Soc el pare del proces %d\n", ret);
     return 0;
  }
}
```

¿Hace falta que el padre espere a que el hijo finalice?

- Es posible que el padre necesite un resultado de su hijo para continuar con su ejecución. ¿Cómo lo conseguimos?
- La función `wait()`
- Ver fork-exec-wait.c

```
int main(void)
{
  int ret;
  char *argv[3] = {"/usr/bin/ls", "-l", NULL};

  ret = fork();

  if (ret == 0) {  // fill
     printf("Soc el fill i el meu id es %d\n", getpid());
     execv(argv[0], argv);
  } else { // pare
     printf("Soc el pare del proces %d\n", ret);
     wait(NULL);
     printf("Torno a ser el pare un cop el fill ha acabat\n");
     return 0;
  }
}
```

¿Siempre el padre debe esperar a que su hijo finalice?

- No necesariamente. El hecho de no utilizar el wait() permiten que el padre e hijo se ejecutan “en paralelo”.
- La línea de comandos nos permite controlar el wait mediante el símbolo **&** al final de la línea.

```
vi fork.c &
vi while-infinit.c
```

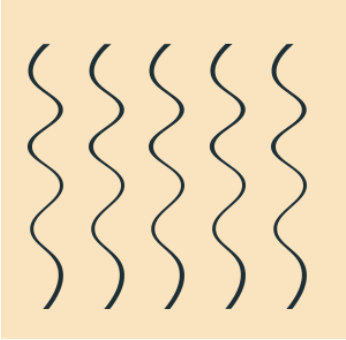
La primera instrucción se ejecuta en paralelo con la línea de comandas. El padre (línea de comandas) no hace ningún wait, y la línea de comandas vuelve a aparecer en seguida. La segunda instrucción hace que el padre haga un wait para que finalice el proceso que se ejecuta. Por ese motivo, vemos que la línea de comandas no vuelve a aparecer hasta que no salimos de la aplicación ejecutada.

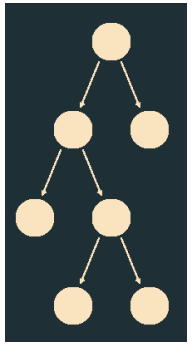
¿Tiene sentido hacer un fork() sin un exec()?

- Si. A **Google Chrome** cada pestaña es un proceso creado con un `fork()`.

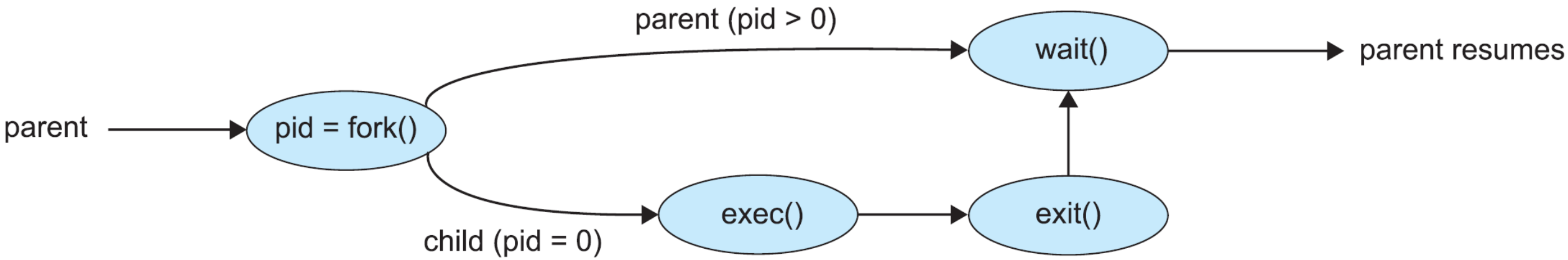
https://blog.chromium.org/2008/09/multi-process-architecture.html

  - Cada proceso se ejecuta en paralelo con el resto de procesos, por lo que las pestañas se cargan en paralelo.
  - Si una página da error, únicamente fallará esta pestaña pero no el resto del navegador web, ya que los procesos son independientes entre sí.
- Otros navegadores como **Firefox** utilizan mecanismos basados en hilos (threads) para obtener el efecto de paralelismo.





Resumen



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Gracias

Los procesos: Gestión del contexto

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Gestión del contexto de los procesos

- Al crear un nuevo proceso mediante `fork()`, el nuevo proceso hijo es una copia exacta del proceso padre, incluyendo su **contexto de ejecución** (datos y recursos asociados a al proceso en el momento dado)
- Un proceso **padre** puede **controlar el contexto del hijo**:
  - Valores en los registros del procesador
  - Privilegios
  - Tiempo máximo de ejecución
  - Memoria máxima que puede ocupar
  - Prioridad para ejecutarse sobre la CPU respecto a otros procesos
  - Descriptores de ficheros abiertos
  - Lugar donde se envía lo que se imprime con un `printf` (salida)
  - Donde recibe lo que se captura con el `scanf` (entrada)
  - Etc.

¿Cómo establecemos el tiempo máximo de un proceso?

- `setrlimit`
- Función del SO que establece límites de recursos para un proceso en ejecución
  - Tiempo CPU
  - Uso de memoria dinámica
  - Uso de la pila
  - Número de ficheros que se pueden abrir
  - Etc.
- Estos límites se aplican solo al proceso actual y a sus procesos hijos, y no afectan a otros procesos
- `getrlimit()` -> nos permite consultar los límites
- El código **while-infinit.c** directamente desde la línea de comandos no se para nunca. La aplicación **fork-exec-setrlimit.c** ejecutará la aplicación anterior limitando el tiempo de ejecución a 1 segundo.

¿Cómo establecemos el tiempo máximo de un proceso?

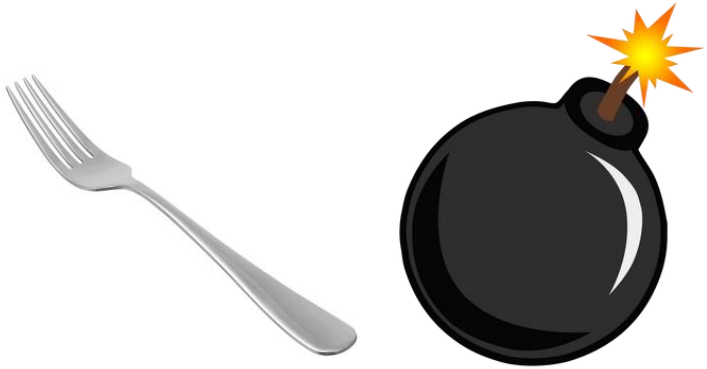
- **`ulimit`**
- Comando de *shell* que establece y consulta los limites de recursos para el *shell* actual y sus procesos hijos
- `ulimit` nos permite:
  - Ejecutar la aplicación **while-infinit.c** limitando el tiempo máximo de ejecución.
  - Podemos ejecutar otras aplicaciones limitando, por ejemplo, el tamaño máximo de la pila.
  - Etc.

```
ulimit -t 2
./while-infinit.c
```

Fork-bomb

- ¿Os atrevéis ahora a ejecutar el fork-bomb limitando algunos de sus recursos?

```
0 #include <stdio.h>
1 #include <unistd.h>
2
3 int main(void)
4 {
5     while (1) { fork(); }
6 }
```



Entrada-Salida

- Además de establecer límites de privilegios sobre los procesos hijos, hay otras cosas que se pueden controlar desde el proceso padre:
  - **Cambio de prioridad**: Asignación de más o menos recursos
  - **Monitoreo de estado**: Monitorear el estado del proceso hijo para saber si está funcionando correctamente, si ha finalizado o si ha ocurrido algún error.
  - **Manejo de señales**: El proceso padre puede enviar señales al proceso hijo para indicarle que realice ciertas acciones (terminar o detenerse temporalmente)
  - **Asignación de recursos**: Asignar recursos adicionales al proceso hijo según sea necesario, como memoria, tiempo de CPU o acceso a dispositivos de entrada y salida (E/S).

Entrada-Salida

- A donde se envía lo que se imprime con un `printf` (salida)
  - Pantalla (salida estándar)
  - Otro proceso
  - Fichero de disco
  - Red
- De donde se recibe lo que se captura con un `scanf` (entrada)
  - Teclado
  - Otro proceso
  - Fichero de disco
  - Red

Gracias

Los procesos: Mecanismos habituales de comunicación

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Comunicación entre procesos

- Hemos visto que el **SO ofrece protección entre procesos**, por lo que los procesos no pueden interferir entre si.
- Sin embargo, los procesos tienen que comunicarse entre si (con permiso del propio SO)
- ¿Cuáles son las técnicas básicas de comunicación entre procesos?
  - **Redirección de E/S estándar**
  - **Tuberías (pipes)**
  - Variables compartidas (shared memory)
  - Señales (signals)
  - Archivos
  - Sockets
  - Semáforos
  - …

printf

- **`printf`**
- Función de la biblioteca estándar de C

```
printf("El valor es %d\n", i);
```

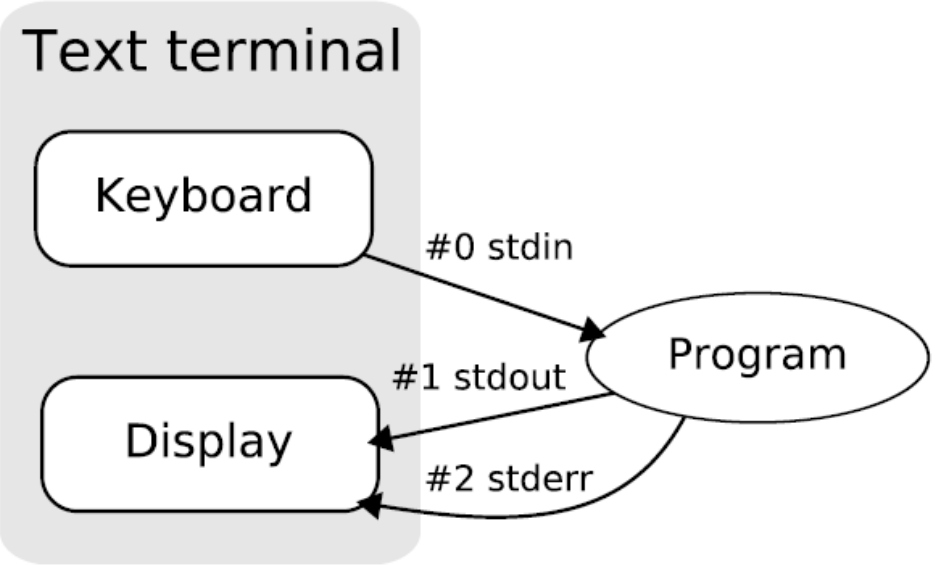
- Esta instrucción genera, en el espacio del usuario, la cadena a imprimir
- Una vez generada la cadena, se realiza una llamada al SO con un `write`

```
count = write (fd, vector, nbytes);
```

- La función `write` (una librería de usuario), realiza la llamada al sistema
  - **`fd`** corresponde con el **descriptor del fichero**, un entero que identifica el dispositivo donde se escribirán los datos.
  - `vector` y `nbytes` hacen referencia al vector de bytes que contiene los datos que se escribirán y el número de bytes a escribir

Descriptores de ficheros (fd)

- ¿Qué `fd` se utiliza para imprimir por pantalla? ¿Y para leer por teclado?
- Todo proceso creado por el SO (Unix o Windows) tiene creado 3 ficheros por defecto
  - Descriptor **0**. "Entrada estándar". Esta asociado por defecto al teclado (se captura con un `scanf`)
  - Descriptor **1**. "Salida estándar". Asociado por defecto a la pantalla del terminar (se imprime con `printf`)
  - Descriptor **2**. "Salida de error". Asociado por defecto a la terminal



Descriptores de ficheros (fd)

- Las instrucciones

```
i = 25;
printf("El valor es %d\n", i);
```

- se convierten en la siguiente llamada al sistema

```
char *vector = "El valor es 25\n";
int nbytes = 15;
count = write(1, vector, nbytes);
```

- Probar el código stdstreams.c. Recordad que se puede rastrear las llamadas a sistema de una aplicación con `strace`.

Gracias

Los procesos: Redirección

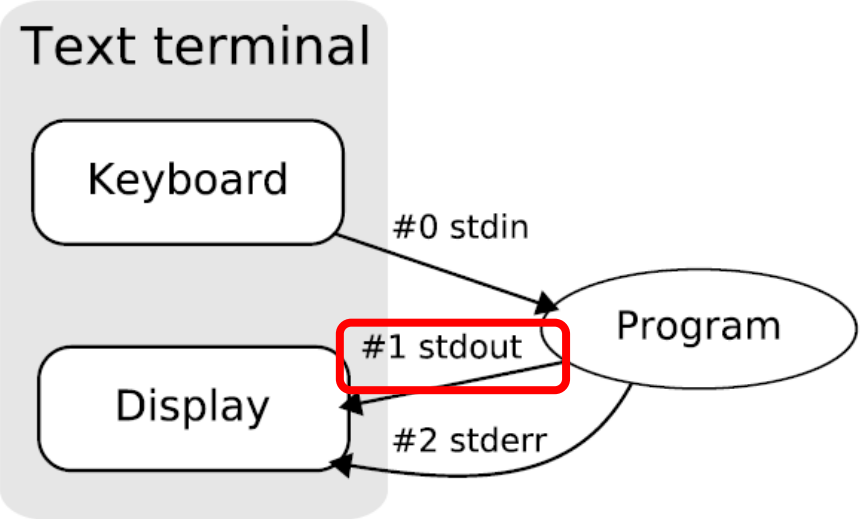
Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

E/S: redirección de la salida estándar a fichero

- Por defecto, **descriptor 1** (`printf`) esta asociado al dispositivo “pantalla”.
- Ahora queremos asociar este descriptor 1 a un **fichero de disco**.
  1. Hacemos un `fork`.
  2. Se abre el fichero donde queremos volcar la información que se escribe al descriptor 1. Realizamos las llamadas al sistema que asocian el fd 1 al fichero.
  3. Ejecutamos el programa. Ahora todo lo que se escribe al fd 1, se escribe a disco.



E/S: redirección de la salida estándar a fichero

- El código fork-redirect-1.c permite hacerlo.

```
int ret;
char *argv[3] = {"/usr/bin/ls", "-l", NULL};

ret = fork();

if (ret == 0) {  // fill
  // obrim fitxer
  int fd = open("fitxer.txt", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
  dup2(fd, 1); // associem fd al descriptor 1
  close(fd);   // tanquem fd
  execv(argv[0], argv); // executem programa
} else { // pare
  ...
}
```

- `dup2(int oldfd, int newfd)`
  - Es una llamada al sistema que se utiliza para duplicar un descriptor de archivo.
  - Duplica el descriptor de archivo `oldfd` en el descriptor de archivo `newfd`
  - Es útil para redirigir la entrada y salida estándar de un proceso

E/S: redirección de la salida estándar a fichero

- El código fork-redirect-1.c permite hacerlo.

```
int ret;
char *argv[3] = {"/usr/bin/ls", "-l", NULL};

ret = fork();

if (ret == 0) {  // fill
  // obrim fitxer
  int fd = open("fitxer.txt", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
  dup2(fd, 1); // associem fd al descriptor 1
  close(fd);   // tanquem fd
  execv(argv[0], argv); // executem programa
} else { // pare
  ...
}
```

- Todo lo que se imprime al descriptor 1 esta asociado al fd (el fichero de disco en vez de la pantalla) -> “se redirige la salida estándar por pantalla a otro fichero”.
- Probar a ejecutar con otros programas como hijo (stdstreams.c)

E/S: redirección de la salida estándar a fichero

- La línea de comandos también permite redirigir la salida estándar a un fichero

```
ls -l > fichero.txt
```

- Probar también

```
./stdstreams > fichero.txt
```

- Una cosa similar pasa con

```
var=$(ls)
```

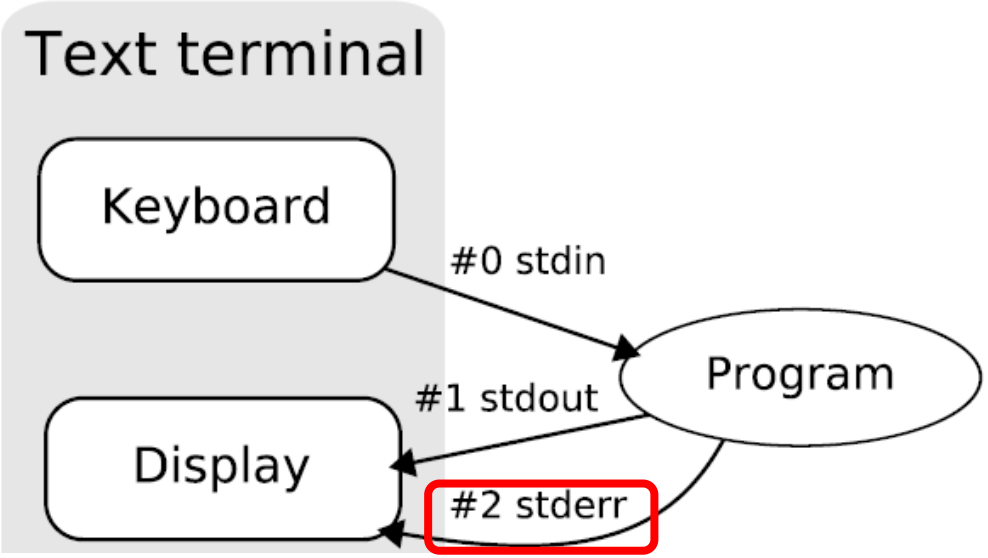
- El intérprete de comandos redirige la salida estándar del comando `ls` (p.ej. a un fichero temporal) para después leerlo y asignarlo a `var`.

```
echo $var
```

E/S: redirección de la salida estándar a fichero

- Al ejemplo anterior capturamos la salida estándar (fd=1), pero no la de **error** (**fd=2**), que es donde se imprimen típicamente los errores.

```
cd info
bash: cd: info: No such file or directory
```



E/S: redirección de la salida estándar a fichero

- Al ejemplo anterior capturamos la salida estándar (fd=1), pero no la de **error** (**fd=2**), que es donde se imprimen típicamente los errores.

```
cd info
bash: cd: info: No such file or directory
```

- Probamos a redirigir la salida estándar

```
cd info > fichero.txt
bash: cd: info: No such file or directory
```

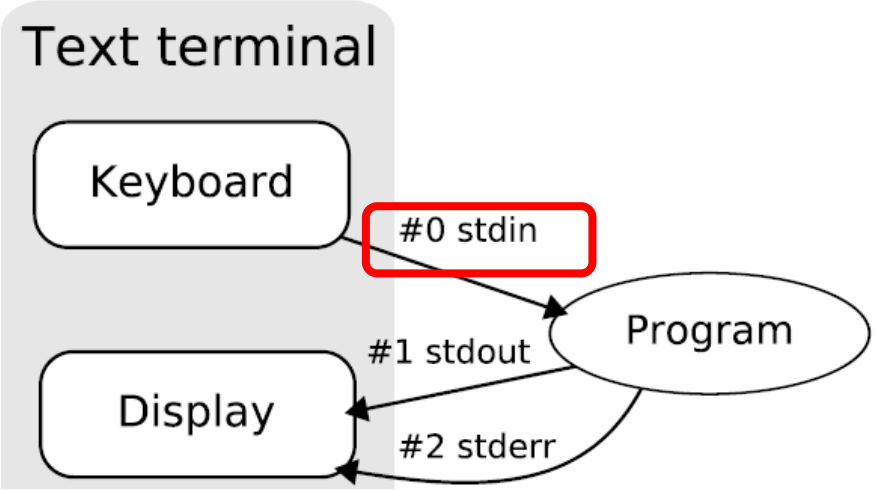
- Podemos capturar la salida del descriptor 2 con

```
cd info 2> fichero.txt
```

- Este comportamiento lo encontramos en otras aplicaciones como el compilador.

E/S: redirección de la entrada estándar a fichero

- Igual que redirigimos la salida estándar (fd=1) a un fichero, podemos redirigir un fichero a la **entrada estándar** (**fd=0**).
- Asociamos el descriptor o (`scanf`) a un fichero de disco.
  1. Hacemos un `fork`
  2. Se abre el fichero desde el cual el descriptor 0 coge los datos
  3. Ejecutamos el programa. `scanf` cogerá los datos de disco.



E/S: redirección de la entrada estándar a fichero

- Ver código fork-redirect-0.c

```
int ret;
char *argv[2] = {"./scanf",  NULL};

ret = fork();

if (ret == 0) {  // fill
  // obrim fitxer
  int fd = open("linies.txt", O_RDONLY);
  dup2(fd, 0); // associem fd al descriptor 0
  close(fd);   // tanquem fd
  execv(argv[0], argv); // executem programa
} else { // pare
  ...
}
```

- Probar a ejecutar primero la aplicación scanf.c directamente desde la terminal. Luego ejecutarla utilizando fork-redirect-0.c

E/S: redirección de la entrada estándar a fichero

- Esta redirección también la podemos hacer desde la línea de comandos

```
./scanf < linies.txt
```

- Podemos incluso redirigir la entrada y salida estándar de una aplicación.

```
./scanf < linies.txt > salida.txt
```

Gracias

Los procesos: Tuberías

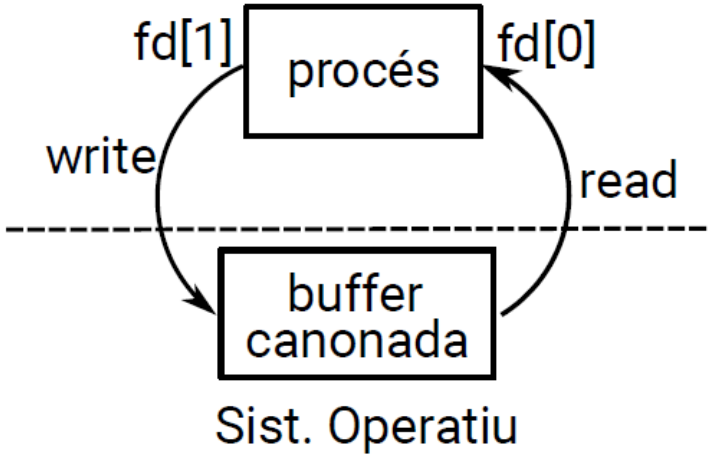
Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Tuberias

- La **tubería** (pipe) es un mecanismo de comunicación entre procesos que permite la transferencia de datos entre un proceso emisor y un receptor.
- ¿Cómo lo implementamos? Llamada al sistema `pipe()` que crea:
  - Un buffer al SO que gestiona la comunicación
  - Dos descriptores de ficheros:
    - Uno de escritura
    - Uno de lectura




```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
  int fd[2];

  if (pipe(fd) == -1) {
    fprintf(stderr, "pipe not created");
    exit(1);
  }

  // fd[1] per escriptura
  // fd[0] per lectura

  return 0;
}
```

Tuberias

- Se puede enviar cualquier tipo de datos por una tubería. Ver pipe.c

```c
int main(void)
{
  int fd[2];
  char buf[30];

  if (pipe(fd) == -1) {
    fprintf(stderr, "pipe not created");
    exit(1);
  }

  printf("writing to file descriptor #%d\n", fd[1]);
  write(fd[1], "test", 4);
  printf("reading from file descriptor #%d\n", fd[0]);
  read(fd[0], buf, 4);
  buf[4] = 0;
  printf("read \"%s\"\n", buf);

  return 0;
}
```

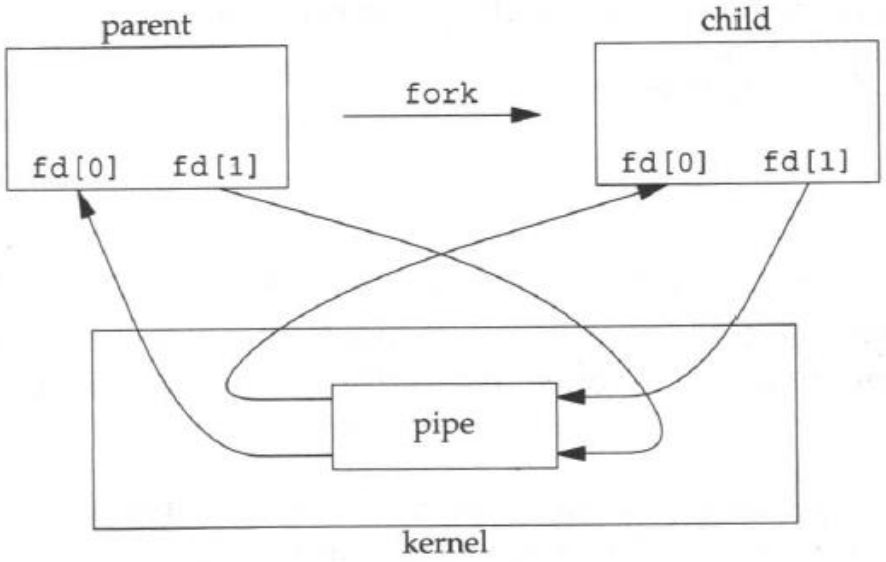
En este ejemplo, antes de imprimir la cadena recibida, hay que poner un byte 0 al final de la cadena (5º byte) ya que el `printf` imprime hasta que encuentra dicho byte 0.

Tuberias

- Las tuberías es canal de comunicación unidireccional entre dos procesos, siempre y cuando estos procesos tengan **relación padre-hijo**.

- P.ej., si el padre escribe a fd[1], el hijo lo podrá leer a través de f[0].

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
  int fd[2];

  pipe(fd);

  if (fork() == 0) { // child
    ...
  } else { // parent
    ...
  }

  return 0;
}
```



Ejemplo de tubería

- Ver código fork-pipe.c (Por simplicidad, no se incluye instrucciones para cerrar las conexiones de las tuberías que no se utilizan)

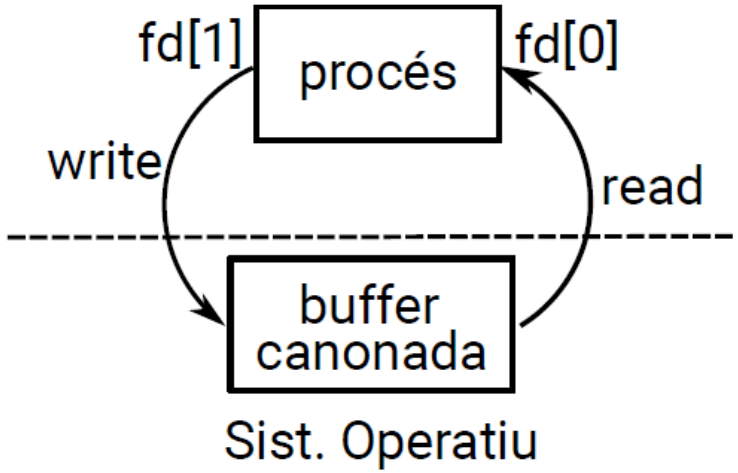
```c
int main(void)
{
  int fd[2];
  char buf[30];

  pipe(fd);

  if (fork() == 0) { // child
    printf("child writing to file descriptor #%d\n", fd[1]);
    write(fd[1], "test", 4);
    exit(0);
  } else { // parent
    printf("parent reading from file descriptor #%d\n", fd[0]);
    read(fd[0], buf, 4);
    buf[4] = 0;
    printf("parent read \"%s\"\n", buf);
    wait(NULL);
  }

  return 0;
}
```

En este ejemplo de tubería

- Un proceso puede introducir datos en la tubería y el otro las va leyendo.
- No hace falta que el primer proceso acabe para que el segundo comience a leer. Es un flujo de comunicación continúa entre los dos procesos
- Al terminal, para realizar operaciones con una tubería se requiere de las funciones `dup2`, así como el cierre de los extremos de la tubería que no se utilizan.
  - fd[1] se asocia a la salida estándar del primer proceso.
  - fd[0] se asocia a la entrada estándar del segundo proceso
- El estándar POSIX requiere que el terminal espere a el último comando de la tubería antes de continuar. Eso implica que el última comando de la tubería será hijo (directo) del *shell* para poder utilizar el `wait` correspondiente.

El buffer de la tubería

- El búfer es un **buffer interno del SO**. Todos los datos pasan por este búfer.
- El búfer es relativamente pequeño (~64 KB)
- Si al leer del búfer no hay ningún dato, el proceso que realiza llamada se bloquea hasta que hay datos disponibles.
- Si al escribir en el búfer, éste se encuentra lleno, el proceso realizar una llamada se bloquea hasta que el otro proceso comienza a vaciar el búfer.



Tuberías

- Típicamente, las tuberías se utilizan en la línea de comandos para combinar múltiples comandos sencillos y conseguir funcionalidades más complejas.

```
ls | ./scanf
```

- Envía el que imprime `ls` por la salida estándar (descriptor 1) a la entrada estándar (descriptor 0) de la aplicación `scanf` (ver código scanf.c). Se trata de un flujo de comunicación continua, mientras `ls` va imprimiendo y enviando la tubería , `scanf` lo va leyendo.
- También podemos combinar las tuberías i redirección

```
ls | ./scanf > salida.txt
```

- Se envia la salida de `scanf` al fichero salida.txt

Gracias

Los procesos: Conclusiones

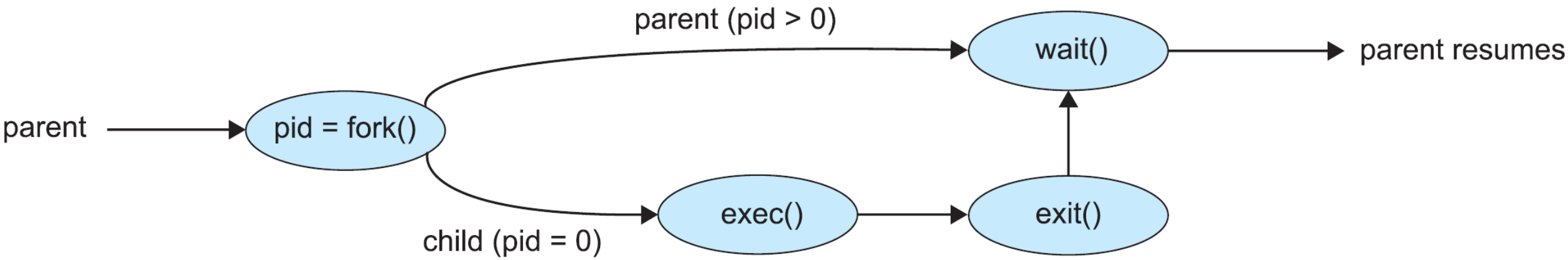
Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Conclusiones

- El SO ofrece una interfaz (llamadas al sistema) para crear procesos.
  - `fork()`
  - `exec()`



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Conclusiones

- Los procesos pueden crear nuevos procesos con un contexto de ejecución limitado (`setrlimit`, `ulimit`)
- Los procesos son independientes entre si.
- El SO ofrece herramientas para que los procesos se puedan comunicar entre sí.
- Los mecanismos mas habituales de comunicación: redirección y tuberías
- En los próximos temas veremos otras herramientas de comunicaciones entre procesos.

Gracias